

**FGA 0238 - Testes de Software – Turma:** 02**Semestre:** 2024.1**Equipe:** 5 - Os Abandonados**Nomes:** João Lucas Pinto Vasconcelos**Matrículas:** 190089601

Atividade 5 – Teste de Segurança

1 Aplicação Analisada

1.1. Identificação da Aplicação

Nome: MEC-Energia**Link para o repositório:** <https://gitlab.com/lappis-unb/projetos-energia/mec-energia>

1.2. Descrição

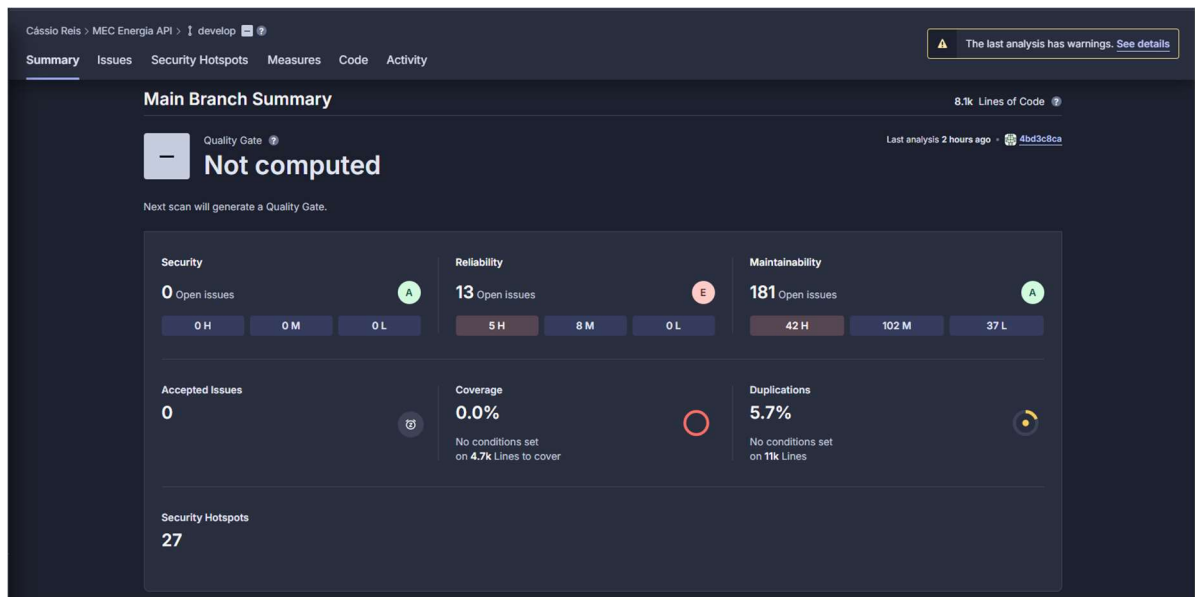
O projeto MEC-Energia é um sistema de recomendação de contratos de energia elétrica, desenvolvido para ajudar as instituições de ensino superior (IES) a gerenciar e avaliar a adequação dos seus contratos de energia. Ele funciona a partir do registro das faturas mensais, permitindo a geração de relatórios que sugerem ajustes nos contratos para otimizar o consumo e economizar recursos.

1.3. Linguagens

MEC Energia Web (*frontend*): TypescriptMEC Energia API (*backend*): Python

2 Visão Geral do Resultado

A análise foi realizada sobre o MEC Energia API (backend). A imagem abaixo mostra a tela do resultado da análise estática utilizando o SonarCloud:



Segundo a análise estática do SonarCloud, o projeto MEC Energia API tem 27 security hotspots, que podem potencialmente introduzir riscos de segurança, e devem ser revisados manualmente para garantir que não representem uma vulnerabilidade.

Além disso, também foram detectados 13 bugs e 181 code smells, que podem impactar a manutenibilidade e a qualidade geral do projeto, exigindo uma revisão cuidadosa e possível refatoração para melhorar o código. Nenhuma vulnerabilidade foi identificada pela ferramenta, o que é um bom sinal para a segurança do projeto.



O SonarCloud está apresentando uma cobertura de testes de 0,0%, embora a cobertura real do projeto seja de 59%.

3 Vulnerabilidades

Como mostrado nas capturas de tela, a ferramenta não detectou vulnerabilidades.

4 Hot Spots

A seguir, tem-se a lista dos security hotspots detectados na análise. A análise identificou os seguintes hotspots, em que os números entre parênteses indicam a quantidade de cada tipo de hotspot identificado:

Authentication (20):

- “password” detected here, review this potentially hard-coded credential. (20)

Cross-Site Request Forgery (CRSF) (1):

- Make sure allowing safe and unsafe HTTP methods is safe here

Permission (2):

- Copying recursively might inadvertently add sensitive data to the container. Make sure it is safe here.
- The python image runs with root as the default user. Make sure it is safe here.

Weak Cryptography (1):

- Make sure that using this pseudorandom number generator is safe here.

Encryption of Sensitive Data (1)

- Using http protocol is insecure. Use https instead

Insecure Configuration (1)

- Make sure this permissive CORS policy is safe here.

Others (1)

- Make sure automatically installing recommended packages is safe here.

5 Análise das Vulnerabilidades ou Hot Spots

Foi averiguado que há um falso positivo no hotspot do tópico de autenticação: **“password” detected here, review this potentially hard-coded credential**, que foi selecionado para análise.

5.1. Authentication

5.1.1. Descrição

“password” detected here, review this potentially hard-coded credential.

Este hotspot indica credenciais hard-coded, como senhas diretamente no código. Essa prática é insegura, e a exposição dessas credenciais é um risco significativo, pois strings hard-coded podem ser facilmente extraídas de código-fonte ou binários, levando a vulnerabilidades conhecidas, como comprometimento de segurança e acesso não autorizado.

5.1.2. Análise de falso positivo:

A instância detectada não se refere a uma credencial armazenada de forma insegura, mas sim a um uso legítimo que não compromete a segurança do sistema.

O security hotspot selecionado para análise se refere a scripts que criam dados iniciais (seed data) para um ambiente de desenvolvimento ou teste, e a senha é usada apenas nesses contextos e não em produção, logo pode ser considerado um falso positivo.

No entanto, boas práticas recomendam evitar hard-coding de senhas sempre que possível e usar métodos alternativos para carregar essas informações de maneira segura.

Descrição do hotspot no SonarCloud:

Because it is easy to extract strings from an application source code or binary, credentials should not be hard-coded. This is particularly true for applications that are distributed or that are open-source.

In the past, it has led to the following vulnerabilities:

- [CVE-2019-13466](#)
- [CVE-2018-15389](#)

Credentials should be stored outside of the code in a configuration file, a database, or a management service for secrets.

This rule flags instances of hard-coded credentials used in database and LDAP connections. It looks for hard-coded credentials in connection strings, and for variable names that match any of the patterns from the provided list.

It's recommended to customize the configuration of this rule with additional credential words such as "oAuthToken", "secret", ...

5.1.3. Solução

Para resolver o hotspot detectado como um falso positivo, é essencial verificar o contexto para garantir que as credenciais detectadas são usadas apenas para fins de teste e não em ambientes de produção. Separar as credenciais de seed data do código-fonte principal, usar dados fictícios ou senhas temporárias para testes, e documentar claramente o propósito dessas credenciais pode ajudar a evitar mal-entendidos futuros.

O SonarCloud recomenda as seguintes práticas:

Recommended Secure Coding Practices

- Store the credentials in a configuration file that is not pushed to the code repository.
- Store the credentials in a database.
- Use your cloud provider's service for managing secrets.
- If a password has been disclosed through the source code: change it.

5.2 Weak Cryptography

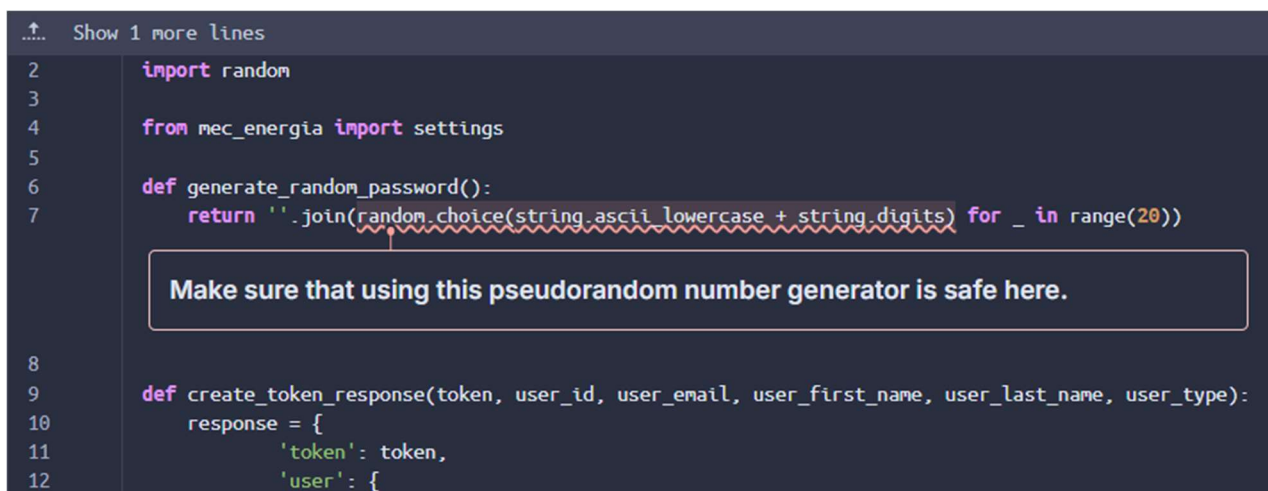
5.2.1 Descrição

Make sure that using this pseudorandom number generator is safe here.

O uso inadequado de geradores de números pseudoaleatórios (PRNGs) é um problema significativo em contextos que exigem criptografia robusta. PRNGs produzem valores baseados em algoritmos determinísticos que não oferecem a aleatoriedade necessária para garantir a segurança criptográfica. Quando PRNGs são usados para gerar valores críticos, como senhas, tokens ou chaves, a previsibilidade desses valores pode ser explorada por atacantes, resultando em graves falhas de segurança. Este problema está classificado como "Cryptographic Failures",

que ocupa o #2 lugar na lista OWASP Top 10 de 2021. Ela abrange falhas relacionadas ao uso inadequado de criptografia, incluindo a escolha de algoritmos e práticas inseguras para proteger dados sensíveis.

5.2.2 Análise de falso positivo



```
... Show 1 more lines
2  import random
3
4  from mec_energia import settings
5
6  def generate_random_password():
7      return ''.join(random.choice(string.ascii_lowercase + string.digits) for _ in range(20))
8
9  def create_token_response(token, user_id, user_email, user_first_name, user_last_name, user_type):
10     response = {
11         'token': token,
12         'user': {
```

Captura de tela do hotspot

Este hotspot de segurança se refere ao uso do módulo random do Python para gerar senhas. O módulo **random** é um gerador de números pseudoaleatórios (PRNG) projetado para tarefas gerais de geração de números aleatórios, mas não é seguro para aplicações criptográficas ou de segurança. O código atual utiliza random.choice() para selecionar caracteres de uma lista, criando senhas previsíveis e menos robustas contra ataques. Isso pode comprometer a segurança das senhas, tornando-as mais vulneráveis a ataques de força bruta e adivinhação, o que pode levar a comprometimentos de contas e dados sensíveis.

Dado que a função generate_random_password() está sendo usada em processos críticos, como a criação de contas de usuário e a redefinição de senhas, o uso de um PRNG não seguro neste contexto constitui uma vulnerabilidade real e não se trata de um falso positivo.

5.2.3. Solução

Para solucionar o problema de segurança relacionado ao uso de geradores de números pseudoaleatórios (PRNGs) inadequados para a geração de senhas, deve-se substituir o PRNG atual por um gerador de números aleatórios criptograficamente seguro. No Python, o módulo secrets é recomendado para esse tipo de operação, pois ele garante que os valores gerados sejam criptograficamente seguros e adequados para contextos que exigem alta imprevisibilidade, como na criação de senhas.

A função `generate_random_password()` deve ser modificada para utilizar o módulo `secrets` em vez de `random`. Isso garantirá que as senhas geradas sejam imprevisíveis e resistentes a ataques de força bruta ou de adivinhação.

Vulnerabilidade solucionada

Apresentação do problema

A mensagem de alerta "Copying recursively might inadvertently add sensitive data to the container" indica que a instrução `COPY . .` no Dockerfile pode estar copiando arquivos e diretórios que não deveriam ser incluídos na imagem do contêiner. Isso pode resultar na inclusão acidental de dados sensíveis, como credenciais, arquivos de configuração ou outros dados que não são necessários para a execução da aplicação em produção.

Problema Identificado

A instrução `COPY . .` copia todos os arquivos e diretórios do contexto de construção para o contêiner, o que pode incluir arquivos indesejados. Isso é especialmente perigoso em ambientes de produção, onde a exposição acidental de dados sensíveis pode levar a vulnerabilidades de segurança.

Solução Proposta

A correção proposta envolve a substituição da instrução `COPY . .` por instruções de cópia mais específicas, que apenas incluem os arquivos e diretórios necessários para a aplicação. Isso minimiza o risco de inclusão de dados sensíveis e melhora a segurança geral da imagem do contêiner.

Código Corrigido

Código Corrigido

Aqui está o Dockerfile corrigido, conforme sua proposta:

```
Dockerfile
1 FROM python:3.10.5
2
3 # -----< COPY >-----
4
5 COPY contracts/ contracts/
6 COPY cronjob/ cronjob/
7 COPY global_search_recommendation/ global_search_recommendation/
8 COPY mec_energia/ mec_energia/
9 COPY recommendation/ recommendation/
10 COPY recommendation_commons/ recommendation_commons/
11 COPY scripts/ scripts/
12 COPY tariffs/ tariffs/
13 COPY tests/ tests/
14 COPY universities/ universities/
15 COPY users/ users/
16 COPY utils/ utils/
17
18 COPY .coveragerc .coveragerc
19 COPY .editorconfig .editorconfig
20 COPY .pylintrc .pylintrc
21 COPY manage.py manage.py
22 COPY pytest.ini pytest.ini
23 COPY requirements.txt requirements.txt
24
25 COPY cronjob/cronjob /etc/cron.d/mec-cron
26
27 RUN apt-get update && \
28 apt-get install -y libpq-dev cron
29
30 RUN pip install -U --no-cache-dir -r requirements.txt
31
32 # -----< cron >-----
33 RUN chmod -R 755 /etc/cron.d/mec-cron && \
34 /usr/bin/crontab /etc/cron.d/mec-cron
35
36 WORKDIR /home/dev/mec-energia-api
```

Justificativa da Correção

Especificidade: Ao especificar quais diretórios e arquivos devem ser copiados, você reduz a chance de incluir arquivos indesejados ou sensíveis.

Segurança: Esta abordagem melhora a segurança da imagem do contêiner, garantindo que apenas os arquivos necessários para a execução da aplicação sejam incluídos.

Manutenção: Um Dockerfile mais explícito e organizado facilita a manutenção e a compreensão do que está sendo incluído na imagem.

Conclusão

A correção proposta para o Dockerfile aborda diretamente o hotspot identificado, garantindo que a aplicação seja construída de forma mais segura. É importante sempre revisar o que está sendo copiado para o contêiner e evitar práticas que possam expor dados sensíveis. Além disso, recomenda-se a utilização de um arquivo `.dockerignore` para excluir arquivos e diretórios que não devem ser incluídos na construção da imagem, como arquivos de configuração, chaves secretas e outros dados

Link para o commit da correção: [Correção de segurança no Dockerfile \(7c7efa1f\) · Commits · João Lucas Vas / MEC Energia API2 · GitLab](#)